Personal Expense & Budget Tracker

GROUP 9 | Rohit Kori (solo - scope reduced, same stack) | Team size: 1 | Duration: 7 days

Backend	Frontend	Database	Auth	Deployment
Flask (REST API)	React + Vite	PostgreSQL + SQLAlchemy	JWT (access + refresh)	Docker Compose

1. Problem Statement

A single-user finance tracker: log income, expenses and transfers across accounts, set monthly budgets per category, and see exactly where the money went.

2. Roles & Permissions

Role	What they can do
User	One role only. Every user sees strictly their own data - no admin matrix to build, so the effort goes into the logic and the dashboard.

3. Data Model

- **User** - credentials, currency, month start day.
- **Account** - name, type (cash / bank / card / wallet), opening balance.
- **Category** - name, kind (income / expense), icon, colour.
- **Transaction** - account, category, type (income / expense / transfer), amount, date, note, transfer_group_id.
- **Budget** - category, month, limit amount.
- **RecurringRule** - template transaction, frequency, next run date, active flag.

4. Functional Requirements

- Account CRUD with a derived running balance.
- Transactions: income, expense and transfer between two accounts.
- Categories with icon and colour, used for grouping and chart colours.
- Filters: date range, account, category, type, text search in notes; pagination.
- Monthly budget per category with budget-versus-actual and an over-budget flag.
- Dashboard: month income/expense/net, spend-by-category pie, 6-month trend line, top categories.
- Recurring transactions generated on login when their next run date has passed.
- CSV export of filtered transactions and CSV import with column mapping.

5. Core Technical Challenge

Identity-scoped queries and balanced transfers. No endpoint ever accepts a user id from the client - the identity comes only from the JWT, and every query filters on it, so changing an id in the URL returns 404, not someone else's data. A transfer writes two linked rows sharing a `transfer_group_id` (a debit and a credit) and the pair is created and deleted together. Account balance is always derived from transactions plus the opening balance, never a stored editable number.

6. React Screens

- Dashboard with KPI tiles and charts (Recharts).
- Transaction table with the filter bar and an add/edit modal.
- Accounts page with balances.
- Budgets page with progress bars per category.
- Settings: categories, recurring rules, CSV import/export.

7. Authentication & Authorisation Specification

- Passwords hashed with bcrypt - plain text passwords must never touch the database.
- Endpoints: `POST /auth/register`, `/auth/login`, `/auth/refresh`, `/auth/logout`, `GET /auth/me`.
- The access token carries a `role` claim; a custom decorator `@role_required("admin")` guards privileged routes on top of `@jwt_required()`.
- Refresh rotation: a refresh call issues a new access token; logout adds the token JTI to a blocklist so it cannot be reused.
- React side: Axios request interceptor attaches the token, response interceptor catches 401, calls refresh once, retries the original request, and logs the user out if refresh also fails.
- Route protection in React by role - a seeker must never be able to open an admin screen, even by typing the URL.

8. Docker Specification

- `backend/Dockerfile` - Python slim base, requirements installed first for layer caching, served by `gunicorn` (not the Flask dev server).
- `frontend/Dockerfile` - multi-stage: Node stage runs `npm run build`, nginx stage serves the static build and proxies `/api` to the backend.
- `docker-compose.yml` - services `api`, `web`, `db` on one network, a named volume for database data, a healthcheck on `db`, and `depends_on` so the API waits for it.
- All secrets (DB URL, JWT secret key) come from `.env`; `.env.example` is committed, `.env` is not.
- Acceptance test: on a clean machine, `git clone` then `docker compose up --build` gives a working app with seeded demo data.

9. Phase-wise Execution Plan

Phase 1 - Day 1: Foundation

- Finalise the data model and draw the ER diagram / collection diagram before writing code.
- Repo and folder structure: `backend/`, `frontend/`, `docker-compose.yml`, `README.md`.
- Docker skeleton up early - database container running and reachable from the API container on day one.
- User model + full JWT auth flow + role seeding. Do not move on until login and refresh work in Postman.

Phase 2 - Day 2 to 3: Core API

- Every core CRUD endpoint with request validation and clear error responses (400 / 401 / 403 / 404 / 409).
- Role guards on every write route; ownership checks so a user can only touch their own records.
- Pagination, filtering and sorting on all list endpoints - never return the whole table.
- Test every endpoint in Postman and save the collection before you write a single React component.

Phase 3 - Day 3 to 5: React Application

- Routing, auth context, protected route wrapper, login and register screens.
- List, detail and create/edit screens for the main entities with client-side form validation.

- Loading, empty and error states on every screen - no blank page while data is fetching.
- Shared Axios instance; no API URL hardcoded in components.

Phase 4 - Day 5 to 6: Core Business Logic

- Implement the core rule described above - this is what separates this project from a CRUD demo.
- Dashboard and aggregation endpoints computed in the database layer, not in JavaScript.
- Search and filters wired end to end.

Phase 5 - Day 6 to 8: Harden and Ship

- Full dockerization verified from a clean clone; fix anything that only worked on your laptop.
- Seed script that creates demo users for each role plus realistic sample data.
- README: architecture overview, setup steps, environment variables, API endpoint table, demo credentials.
- Postman collection, bug fixing pass, and a 5-minute demo recording walking through each role.

10. Work Split

Owner	Responsibility
Solo plan	Day 1 schema + auth + Docker skeleton. Day 2 accounts and transactions API. Day 3 budgets, recurring rules, report endpoints. Day 4-5 React dashboard and transactions. Day 6 budgets screen, CSV import/export. Day 7 Docker, seed, README, demo.

Every member must commit backend code and review at least one pull request - no single member owns the whole repository.

11. Deliverables

- GitHub repository containing both applications with a readable commit history from every member.
- README.md with setup instructions, environment variables and an API endpoint table.
- ER diagram (SQL) or collection diagram (MongoDB).
- Seed script with demo data and demo credentials per role.
- Postman collection covering every endpoint.
- Working `docker compose up --build` on a clean machine.
- 5-minute demo video.

12. Evaluation

Area	Weight	What is checked
Functionality	30%	All listed features work end to end without manual database edits
API design & JWT	20%	Correct status codes, validation, role guards, refresh and blocklist working
Core business logic	20%	The project-specific core rule is correctly and safely implemented
React frontend	15%	Component structure, state handling, validation, loading and error states
Docker & docs	15%	One-command startup, clean compose file, complete README and seed data

13. Stretch Goals (only after everything above works)

- Dark mode.
- Month-over-month comparison view.